



Lecture[1]

C FUNDAMENTALS



Agenda

- Data type and storage
- Expression
- Control flow



Agenda

- Data type and storage
- Expression
- Control flow



Data Type

- C has the following simple data types:

Data type	C code	Size in bytes	Range
Char or signed char	char	1	-128 to 127
Unsigned char	unsigned char	1	0 to 255
int or signed int	int	at least 2	-32768 to 32767
Unsigned int	unsigned int	at least 2	0 to 65535
Short int or signed short int	short	2	-32768 to 32767
Unsigned short int	unsigned short	2	0 to 65535
Long int or Signed long int	long	at least 4	-2147483648 to 2147483647
Unsigned long int	unsigned long	at least 4	0 to 4294967295
float	float	4	3.4E-38 to 3.4E+38
double	double	8	1.7E-308 to 1.7E+308
Long double	long double	10*	3.4E-4932 to 1.1E+4932

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6
7      int b=3;
8      //print out the value of b
9      printf("the size of b is %u", sizeof(b));
10     return 0;
11 }
```

the size of b is 4

On 32-bit machine is
usually 4 bytes (32bits)



Unsigned int

- How to store an unsigned int
 - All bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1111_2 = 15_{10}$$

– 2 Bytes = 16 bits

– $0 \sim 2^{16} - 1 = 65535$

```
#include <stdio.h>
#include <limits.h>

int main() {
    unsigned int a = 3000000000U;
    unsigned int b = 2000000000U;
    unsigned int sum = a + b;

    printf("a = %u\n", a);
    printf("b = %u\n", b);
    printf("a + b = %u (expected: 5000000000)\n", sum);
    printf("actual result = (a + b) mod 2^32 = %u\n", sum);

    return 0;
}
```



int with sign

- Need to both represent positive and negative int
- Sign and magnitude (原码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Other bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$00001111_2 = 15_{10}$$

– But

- What about 0? Two representations
- $15 + (-15) = ?$

1	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$10001111_2 = -15_{10}$$



One's-Complement for signed numbers

- One's-Complement (二进制反码)

- Use highest bit for sign: 0 -> +, 1 -> -
- Positive number X is encode of X
- Negative number X is encode $\sim X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

– $-15 + (-15) =$

– Still two representation of '0'

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

-15_{10}



Two's-Complement for signed numbers

- Two's-Complement (二进制补码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Positive number X is encode of X
 - Negative number X is encode of $2^n + X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

(方法2: 反码+1)

1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

—

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

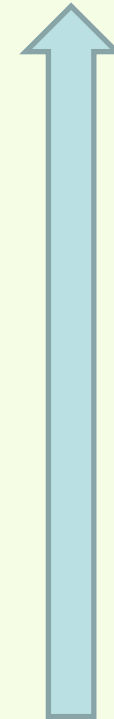
-15_{10}

1	1	1	1	0	0	0	1
---	---	---	---	---	---	---	---



Two's-Complement for signed numbers

- Only one 0
- $-2^{n-1} \sim 2^{n-1}-1$
- The order is respected
- Arithmetic calculation



Two's complement	Decimal
[0]111	+7
[0]110	+6
.....	
[0]001	+1
[0]000	0
[1]111	-1
.....	
[1]001	-7
[1]000	-8



Integer Overflow and Handling

- Signed integer overflow is even more critical than unsigned integer because it may cause **sign reversal**
- In scientific computing, integer overflow can cause catastrophic consequences
 - Timeline chaotic; wrong memory regions; etc.

Strategy:

- Check overflow
- Use GCC's '___builtin_add_overflow' series functions

```
int main() {  
    int a = INT_MAX;  
    int b = 1;  
    int result;  
  
    if (___builtin_add_overflow(a, b, &result)) {  
        printf("Error: Addition overflowed!\n"); // 这行会被执行  
    } else {  
        printf("The sum is: %d\n", result);  
    }  
  
    // 即使溢出, result 仍然被赋值了 (通常是回绕后的值)  
    // 但我们不应该使用它, 因为我们已经知道它溢出了。  
    printf("The (invalid) value in result is: %d\n", result); // 可能输出 -2147483648  
  
    return 0;  
}
```



Floating types

float

single-precision floating-point

double

double-precision floating-point

long double

extended-precision floating-point

Type	Smallest Positive Value	Largest Value	Precision	Size
float	1.17×10^{-38}	3.40×10^{38}	7 digits	4 Bytes
double	2.22×10^{-308}	1.79×10^{308}	15 digits	8 Bytes

```
float x;  
scanf("%f", &x);  
printf("%f", x);
```

```
double x;  
scanf("%lf", &x);  
printf("%f", x);
```

```
long double x;  
scanf("%Lf", &x);  
printf("%Lf", x);
```



How to represent a float

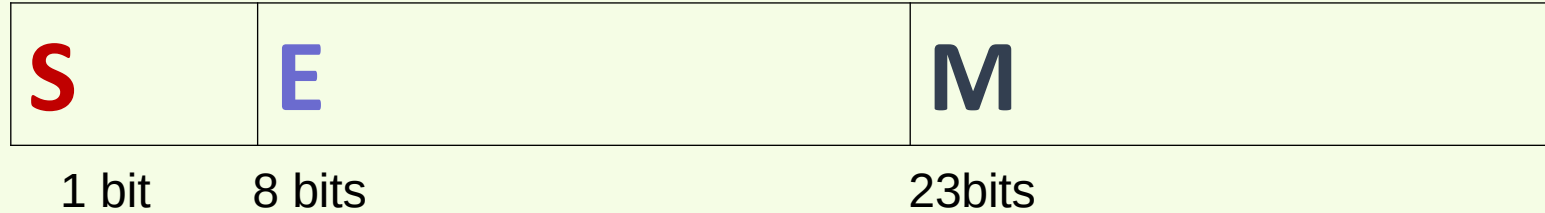
- How to represent a decimal fraction in binary?
 - $-10.125 \rightarrow 1010.001_2$
- Formatted form
 - $1010.001_2 = 0.1010001_2 \times 2^{100}_2$
- Encoding of float
 - $N = (-1)^S \times 0.M \times 2^E$

S	E	M
0	000100	001010001

10.125 in 16 bits



IEEE 754 float

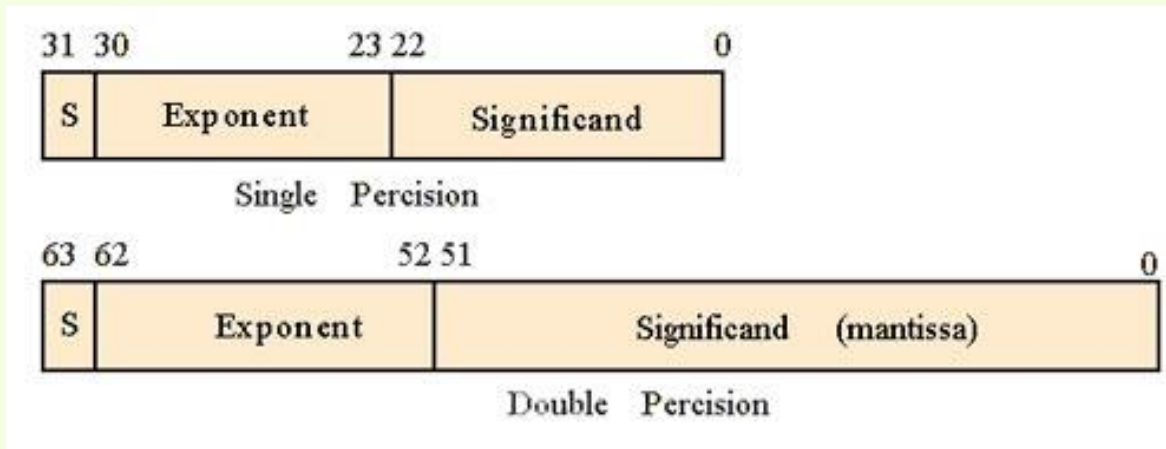


- M is consider as 0.1xxxx->1.xxxx
 - Save one bit
- E use $E = E_{\text{real}} + 127$
- $N = (-1)^S \times 1.M \times 2^{E-127}$
- The range is
 - Smallest value: $E=1, M=0, N_{\text{min}} = 1.0 \times 2^{1-127} = 2^{-126}$
 - Biggest value: $E=254, M=11\dots1,$
 $N_{\text{max}} = 1.11\dots1 \times 2^{254-127} = (2-2^{-23}) \times 2^{127}$



Float/double precision

- Float and double are not precise (with finite precision)
 - The storage is not accurate
 - Floating operation uses specific hardware
- Use double if no specific requirement





Floating-Point Precision Issues

- The finite precision of floating-point numbers is one of the challenge in scientific computing: one small quantization error might accumulate during operation.
- Absorption phenomenon:
 - Example: $1e8f + 1.0f$ in single precision, 1 will disappear, because 1 is too small relative to 10^8 in the finite mantissa bits

```
float large = 1e8f;  
float small = 1.0f;  
float sum1 = large + small - large;  
float sum2 = large - large + small;  
  
printf("Precision loss demonstration:\n");  
printf("(1e8 + 1) - 1e8 = %.10f (expected: 1.0)\n", sum1);  
printf("1e8 - 1e8 + 1 = %.10f (expected: 1.0)\n", sum2);
```

Output

Precision loss demonstration:

$(1e8 + 1) - 1e8 = 0.0000000000$ (expected: 1.0)
 $1e8 - 1e8 + 1 = 1.0000000000$ (expected: 1.0)



Handling accumulation errors

- Idea: to record the rounding error through operations:

for $s = a + b$, the rounding error $\epsilon = (a + b) - s$

- Kahan summation algorithm:

```
// Kahan summation vs naive summation
float naive_sum = 0.0f;
float kahan_sum = 0.0f;
float c = 0.0f; // Compensation term

for(int i = 0; i < 10000000; i++) {
    naive_sum += 0.01f;

    float y = 0.01f - c;
    float t = kahan_sum + y;
    c = (t - kahan_sum) - y;
    kahan_sum = t;
}
```

Accumulating 0.01 ten million times:
Naive summation: 100958.343750 (error: 958.343750)
Kahan summation: 100000.000000 (error: 0.000000)

- Numerical Stability in Scientific Computing*



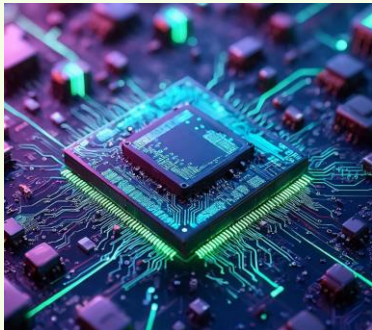
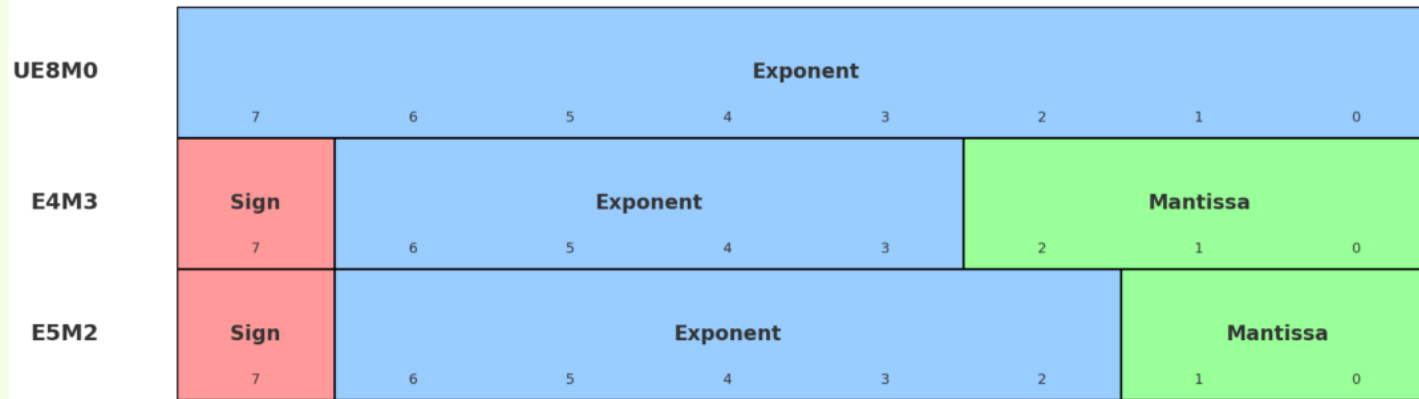
DeepSeek & UE8M0 FP8



DeepSeek 作者 3天前

3109

UE8M0 FP8是针对即将发布的下一代国产芯片设计。



虽然下一代国产芯片虽然已经在为FP8做出准备，但HBM/LPDDR 带宽仍然与顶尖芯片存在较大差距。而UE8M0让一组32个FP8数据只追加8bit缩放引子，相比传统的4B（32bit）FP32缩放直接节省75%的流量，这种空间节约措施被视作下一代架构的重要优化方向。

Precision?

UE8M0, through exponential representation, provides logarithmically uniform quantization levels across the entire range, dealing with long tail characteristics of weight distribution in LLM.

Furthermore, DeepSeek overcomes UE8M0's precision limitations through clever mixed-precision strategies. They identify critical layers in the model maintain FP16 precision, while quantizing other layers to FP8. Quantization-Aware Training is used.



Character Types

- Char is also an integer type

- 'a', '1'

- %c in scanf and printf

- ASCII code

- '1'的ASCII编码是49,
- 当变量a==49, a的值就是'1'

```
char ch;  
int i;  
i = 'a';           // i is now 'a', its value is 97  
ch = 65;           // ch is now 'A', its value is 65  
ch = ch + 1;       // ch is now 'B', its value is 66  
ch++;              //ch is now 'C', its value is 67
```

```
for(ch = 'A'; ch <= 'Z'; ch++)  
{  
    printf("%c\n",ch);  
}
```



Boolean

- Boolean: {true, false}
- There is no Boolean data type in C
- Any integer $\neq 0$ is considered true
- 0 is considered false

```
int a= 3;
```

```
int b = !a; // b=0
```



Constants int

- Integer constants:
 - Decimal: 11,12345
- Declare a named constant:
 - Using **const** keyword

```
const int PAIR=2
```

- Using **#define** pre-processor directive

```
#define PAIR 2
```

```
int main()
{
    int n;
    const int TIMES = 2;
    scanf("%d",&n);
    printf("Output is %d\n",TIMES*n);
}
```



Constants

- Character constants
- String constants
 - "I am a string"
 - Character array
 - always null ('\0') terminated.
 - (see <string.h> for string functions)



Agenda

- Data type and storage
- Expression
- Control flow



Character set in C

Get a good Compiler, it
will tell you

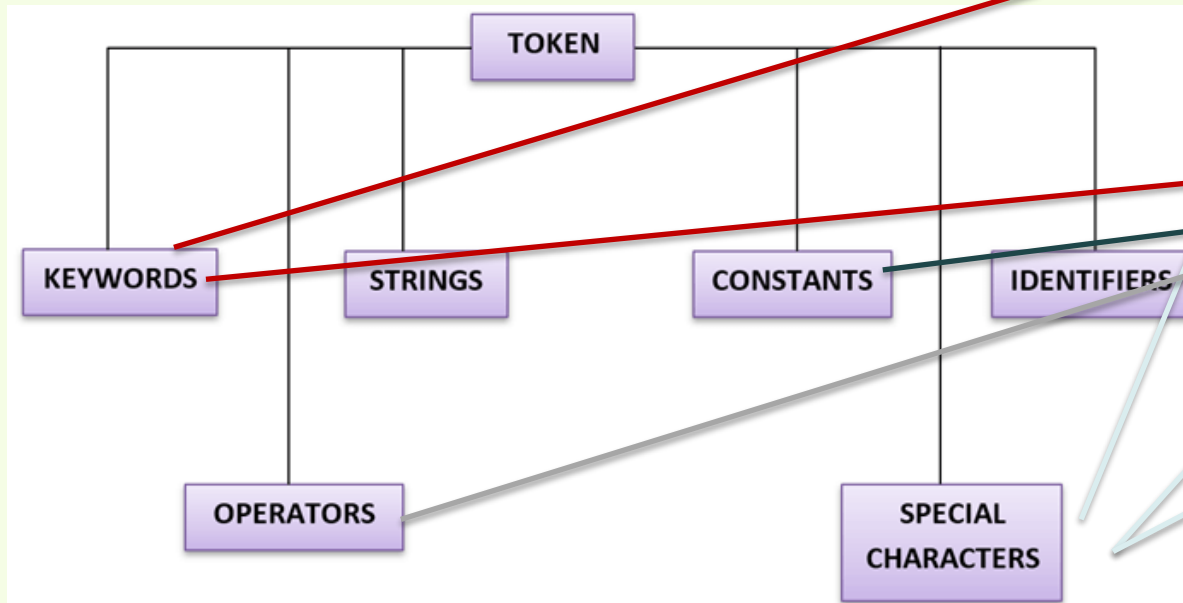
- 'C' has its own character set
 - Letters : A-Z, a-z
 - Numbers: 0-9
 - White spaces
 - Blank space
 - New line
 - Carriage return
 - Horizontal tab
 - Special characters

, (comma)	{ (opening curly bracket)
. (period)	} (closing curly bracket)
; (semi-colon)	[(left bracket)
: (colon)	] (right bracket)
? (question mark)	((opening left parenthesis)
' (apostrophe)	) (closing right parenthesis)
" (double quotation mark)	& (ampersand)
! (exclamation mark)	^ (caret)
(vertical bar)	+ (addition)
/ (forward slash)	- (subtraction)
\ (backward slash)	* (multiplication)
~ (tilde)	/ (division)
_ (underscore)	> (greater than or closing angle bracket)
\$ (dollar sign)	< (less than or opening angle bracket)
% (percentage sign)	# (hash sign)



Token in C

- Token is the smallest unit in C program
 - The compiler breaks a program into tokens and proceeds them



```
#include <stdio.h>

int main() {
    int a,b;
    printf("input two integers\n");
    scanf("%d %d",&a, &b);
    printf("A+B =%d\n",a+b);
    return 0;
}
```



Keywords & identifier

Get a good Compiler, it
will tell you

- Keywords have fixed meanings
– (in lowercase letters)

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	short	float	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

- An identifier is a name assigned to an element in a program
 - Name of a variable/function
 - Formed with only letters/numbers/underscore
 - Begin with an alphabet or an underscore
- Packet_Size, underAmour



Keywords & identifier

- A syntax error will produce at compilation

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int x=1;
6
7      ↪ printf("x=%d",x);
8      return(0);
9  }
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int if=1;
6
7      ↪ printf("if=%d",if)
8      return(0);
9  }
```

```
motclef2.c: In function 'main':
motclef2.c:5:9: error: expected identifier or
↪  '(' before 'if'
    int if=1;
        ^
```



Declaration, Assignment & Initialization

- Declaration (声明)

`int count;`
`char char_a, char_b;`

类型不能被改变

- Assignment (赋值)

`count = 0;`

- Initialization (初始化)

<类型名称> <变量名称> = <初始值>

`int count = 0;`
`char char_a = 'a';`

```
int main()
{
    int x;
    int y;
    x=26;
    y= x+10;
    printf("%d\n",y);
    return 0;
}
```

变量务必初始化



C99 vs ANSI C

```
int main()
{
    int x=0;
    int y=0;
    scanf("%d", &x);
    y= x+10;
    int z = x+y;
    printf("%d\n",z);
    return 0;
}
```

C99

```
int main()
{
    int x=0;
    int y=0;
    int z=0;
    scanf("%d", &x);
    y= x+10;
    z = x+y;
    printf("%d\n",z);
    return 0;
}
```

ANSI C



Expressions

- Arithmetic operator:

$+, -, *, /, \%$

- Relation operators:

$<, >, <=, >=, ==, !=$

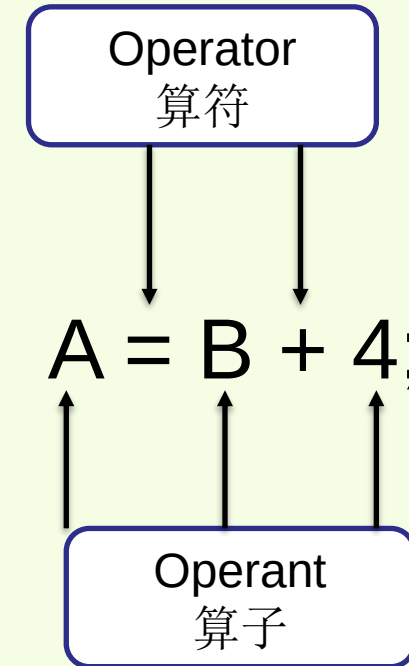
- Assignment operators: $+=, -=, /= \dots$

- Increment and decrement operators:

$++, --, ++, --$

- Logical operator:

$\&\&$ (and), $\|\|$ (or)





Relation Operators

- Relation operators has lower precedence than arithmetic operators:
 - , >=, <, <=
 - ==, !=
- Don't confuse = and == ! The compiler will warn “suggest parens”.

```
int x=5;
if (x==6)    /* false */
{
    /* ... */
}
/* x is still 5 */
```

```
int x=5;
if (x=6)    /* always true */
{
    /* x is now 6 */
}
/* ... */
```



Assignment Operators

- Most binary operators have a corresponding assignment operator *op*
 - $+, -, *, /, \%, <<, >>, \&, \wedge, |$
 - $expr1\ op = expr2 \Leftrightarrow expr1 = (expr1)\ op\ (expr2)$
 - $x\ *=\ y+1 \Leftrightarrow x = x * (y+1)$
- Assignment statement has a value
- Most used one 'sum += value' in numerical computation



Increment and Decrement Operators

<code>x++</code>	post-increment <code>x</code>	<code>++x</code>	pre-increment <code>x</code>
<code>x--</code>	post-decrement <code>x</code>	<code>--x</code>	pre-decrement <code>x</code>

Note the difference between `++x` and `x++`:

```
int x=5;
int y;
y = ++x;
/* x == 6, y == 6
*/
```

```
int x=5;
int y;
y = x++;
/* x == 6, y == 5
*/
```

Modern compiler can usually optimize away performance difference.
But it is recommended the prefix form, especially in C++



Bitwise Operations

- Applied to char, int, short, long
 - And $\&$
 - Or $|$
 - one's complement $\sim$
 - Exclusive Or $\wedge$
 - Left-shift $\ll$
 - Right-shift $\gg$



Bitwise Operations

Operator	Typical Usage	Express
& (AND)	1. Take one specific bit	$n \& 0x10$; (5 th bit)
	2. Put several bits to 0 while keep others	$n = n \& 0xfffff00$; (lower 8 bits to 0)
(OR)	1. Put several bits to 1 while keep others	$n = n 0xff$; (lower 8 bits to 1)
~ (Complement)	1. Put all bits to complement (not “+ to –”)	$n = \sim n$; ($\sim 1 \rightarrow -2$)
^ (XOR)	1. Put one bit or a set of bits to complement	$n = n \wedge 0x10$; (5 th bit turns to its complement)
	2. Switch two values	$a = a \wedge b$; $b = a \wedge b$; $a = a \wedge b$;
<<	1. Equivalent to $*2^i$	$n \ll i$;
>>	2. Equivalent to $/2^i$	$n \gg i$;



Logical Operators

- Logical operators (&&, ||, !)
- They have short-circuit evaluation characteristics
 - In ``A && B``, if A is false, B will not be evaluated;
 - in ``A || B``, if A is true, B will not be evaluated.



Conditional Expressions

- Conditional expressions
- `expr1? expr2:expr3;`
- if `expr1` is true then `expr2` else `expr3`

```
(u%2==0)?printf("even num\n"):printf("odd num\n");
```



Operator Precedence and Associativity

highest: + - (unary)

* / %

lowest: + - (binary)

$$-i * -j = (-i) * (-j)$$

$$+i + j / k = (+i) + (j / k)$$

left associative: it groups from left to right

right associative: it groups from right to left

The binary arithmetic operators (*, /, %, + and -) are all left associative (from left to right)

$$i - j - k = (i - j) - k \quad i * j / k = (i * j) / k$$

The unary arithmetic operators(+ and -) are both right associative

$$- + i = - (+i)$$



Expression Evaluation

Precedence	Name	Symbol(s)	Associativity 结合关系
1	X++/X--		Left (from left to right)
2	++X/--X unary +/- (单目)		Right (from right to left)
3	multiplicative	*, /, %	left
4	additive	+, -	left
5	logical	&&,	left
6	assignment	=, *=, /=, +=, -=	Right



Expression Evaluation examples

```
int a,b,c,d,e,f;  
a=1;   b=2;   c=3;  
d = !(a+b)+c-1&&b+c/2;  
e = a--&&b++&&++c;  
f = a+=b++-c*d ;
```



Quiz 0



Task 1: The Height of LBJ

- What's the height of LBJ
- How 6 feet 9 inches 2.06m?
- **Meter = (foot + inch/12)*0.3048**
- Write a program

```
int main()
{
    printf("input foot and inch:\n");
    int foot;
    int inch;
    ...
    return 0;
}
```



				HEIGHT 6'9" (2.06m)	WEIGHT 250lb (113kg)
PPG 25.0	RPG 7.7	APG 7.8	PIE 19.1	AGE 36 years	BIRTHDATE December 30, 1984



Task 2: A+B

1. Most used type
2. Reading and Writing Integers
3. Formatted Input/Output
4. A+B